**Experiment No.: 09**

**Name of the Experiment:** Smart Line Following and Obstacle Avoidance Robot

**Objectives:**

1. To design and construct a robot that autonomously follows a black line on a white surface using IR sensors.
2. To implement obstacle detection using an HC-SR04 ultrasonic sensor mounted on a servo motor, and program an avoidance manoeuvre that scans the left and right sides and passes the obstacle on the side with more free space.
3. To control two DC motors with PWM speed control and bidirectional drive using an L293D-based motor driver.

**Theory:**

A line-following robot is an autonomous mobile robot that detects and navigates along a predefined path, typically a black line on a white surface. The robot uses Infrared (IR) sensors to distinguish the high-reflectivity white surface from the low-reflectivity black line. When an IR sensor is positioned over the black line, the reflected infrared light is minimal and the sensor output goes HIGH; when it is positioned over the white surface, more infrared light is reflected and the output is LOW. The two sensors are placed on either side of the line, so the robot moves forward while both sensors see the white surface and steers back toward the line as soon as one of them reaches the black line. By comparing the states of the left and the right IR sensor, the microcontroller determines the steering correction needed to keep the robot on the line.

The obstacle avoidance feature is implemented using an HC-SR04 ultrasonic sensor, which emits an ultrasonic pulse and measures the time taken for the echo to return. The distance to an obstacle is calculated using the formula:

**Distance (cm) = (Echo Duration (µs) × 0.034) / 2**

When the measured distance falls to the preset threshold (5 cm in this experiment) or below, the robot halts. The ultrasonic sensor is mounted on an SG90 servo, which turns it to the right and then to the left to measure the free distance on each side. The robot then turns toward the side with more free space, passes the obstacle with a fixed sequence of turns and forward moves, and resumes line following afterward.

Motor control is handled by an L293D-based motor driver. For each motor, one enable pin receives a PWM signal from the Arduino to set the speed (analogWrite), and two direction pins set forward, backward or stop. The speed is set by the constant MOTOR_SPEED (110 out of 255), and a turn is made by driving the two wheels in opposite directions. The SG90 servo is driven by pulses generated in software, so no servo library is used. The measured distances are also printed on the Serial Monitor at 9600 baud.

**Required Apparatus:**

**Table.9.1: Required Apparatus**

| Sl. No. | Components | Specification | Quantity |
|---|---|---|---|
| 1. | Arduino Uno | ATmega328P, 5V, 16 MHz | 1 |
| 2. | Motor Driver Module | L293D based, dual H-bridge with enable pins | 1 |
| 3. | DC Motors with Wheels | 6V, with rubber tyres | 2 |
| 4. | IR Sensor Module | Digital output, 3-pin, adjustable sensitivity | 2 |
| 5. | Ultrasonic Sensor | HC-SR04, 2 cm – 400 cm range | 1 |
| 6. | Mini Servo Motor SG90 | 4.8~6 V | 1 |
| 7. | Li-ion Battery Pack | 3.7 V, 2000 mAh | 3 |
| 8. | Robot Chassis | 2-layer acrylic platform | 1 |
| 9. | Jumper Wires | Male-to-male, male-to-female | As required |
| 10. | USB Cable | Type-A to Type-B, for programming | 1 |

**Circuit Diagram:**

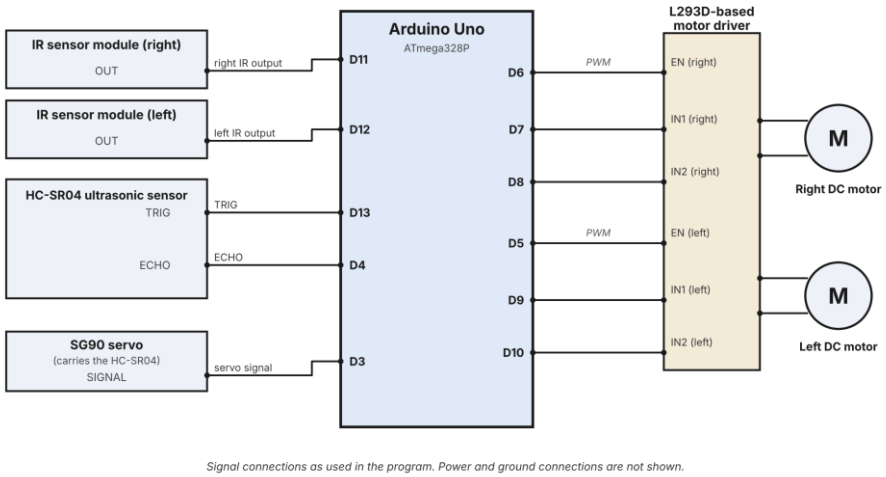



**Fig.9.1:** Signal connection diagram of the LFR with obstacle detection and avoidance (pins as used in the program)

**Experimental Setup:**

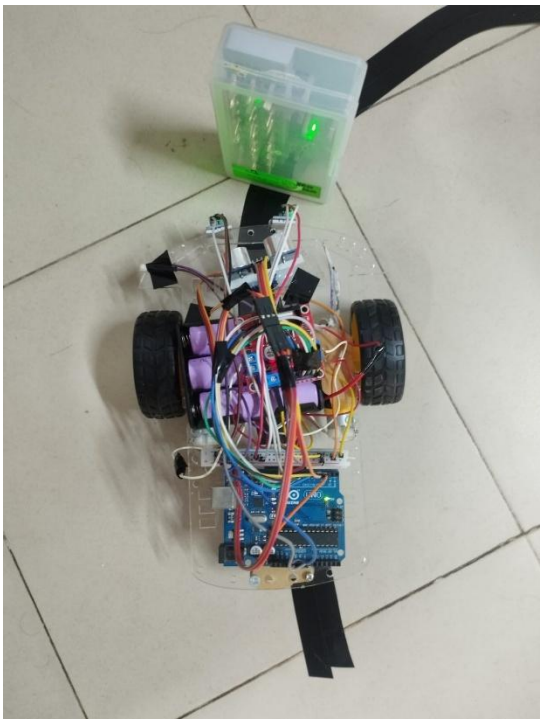


**Fig.9.2:** Experimental setup for assembled line-following robot

**Arduino Program:**

```
// Line Follower with Obstacle Avoiding Robot
// Uses original pin layout with video-style servo sweep & obstacle logic

#define IR_SENSOR_RIGHT 11
#define IR_SENSOR_LEFT  12
#define MOTOR_SPEED     110

// Right motor
int enableRightMotor = 6;
int rightMotorPin1   = 7;
int rightMotorPin2   = 8;

// Left motor
int enableLeftMotor  = 5;
int leftMotorPin1    = 9;
int leftMotorPin2    = 10;

// Servo + HC-SR04
#define SERVO_PIN   3
#define TRIG_PIN    13
#define ECHO_PIN    4

#define OBSTACLE_DIST_CM 5 // Set distance threshold (cm)

int distance_L, distance_F, distance_R;
```

```
void setup()
{
  Serial.begin(9600);

  TCCR0B = TCCR0B & B11111000 | B00000010;

  pinMode(enableRightMotor, OUTPUT);
  pinMode(rightMotorPin1,   OUTPUT);
  pinMode(rightMotorPin2,   OUTPUT);

  pinMode(enableLeftMotor,  OUTPUT);
  pinMode(leftMotorPin1,    OUTPUT);
  pinMode(leftMotorPin2,    OUTPUT);

  pinMode(IR_SENSOR_RIGHT, INPUT);
  pinMode(IR_SENSOR_LEFT,  INPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(SERVO_PIN, OUTPUT);

  // Sweep servo to center (70 degrees) on startup
  for (int angle = 0;   angle <= 50;  angle += 5) { servoPulse(SERVO_PIN,
angle); }
  for (int angle = 50;  angle <= 100; angle += 5) { servoPulse(SERVO_PIN,
angle); }
  for (int angle = 100; angle >= 50;  angle -= 5) { servoPulse(SERVO_PIN,
angle); }

  distance_F = Ultrasonic_read();

  rotateMotor(0, 0);
  delay(500);
}

void loop()
{
  //================================================
  //      Line Follower and Obstacle Avoiding
  //================================================

  distance_F = Ultrasonic_read();
  Serial.print("D F="); Serial.println(distance_F);

  int rightIR = digitalRead(IR_SENSOR_RIGHT);
  int leftIR  = digitalRead(IR_SENSOR_LEFT);
```

```
  // Both sensors on white line -> go forward (check obstacle first)
  if (rightIR == LOW && leftIR == LOW)
  {
    if (distance_F > OBSTACLE_DIST_CM) { rotateMotor(MOTOR_SPEED, MOTOR_SPEED);
}
    else                               { Check_side(); }
  }
  // Right sensor on black -> turn right
  else if (rightIR == HIGH && leftIR == LOW)
  {
    rotateMotor(-MOTOR_SPEED, MOTOR_SPEED);
  }
  // Left sensor on black -> turn left
  else if (rightIR == LOW && leftIR == HIGH)
  {
    rotateMotor(MOTOR_SPEED, -MOTOR_SPEED);
  }
  else
  {
    rotateMotor(0, 0);
  }

  delay(100);
}

// ---------------------------------------------------------
// Servo pulse (software servo, no Servo.h needed)
// ---------------------------------------------------------
void servoPulse(int pin, int angle)
{
  int pwm = (angle * 11) + 500; // Convert angle to microseconds
  digitalWrite(pin, HIGH);
  delayMicroseconds(pwm);
  digitalWrite(pin, LOW);
  delay(500); // Refresh cycle of servo
}

// ---------------------------------------------------------
// Ultrasonic distance read
// ---------------------------------------------------------
long Ultrasonic_read()
{
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
```

```
  long duration = pulseIn(ECHO_PIN, HIGH, 25000);
  if (duration == 0) return 999;
  return duration * 0.034 / 2;
}

// ---------------------------------------------------------
// Scan left & right, then decide which way to turn
// ---------------------------------------------------------
void Check_side()
{
  rotateMotor(0, 0);
  delay(1000);

  // Sweep right (70 -> 140)
  for (int angle = 50; angle <= 100; angle += 5) { servoPulse(SERVO_PIN,
angle); }
  delay(3000);
  distance_R = Ultrasonic_read();
  Serial.print("D R="); Serial.println(distance_R);
  delay(1000);

  // Sweep left (140 -> 0)
  for (int angle = 100; angle >= 0; angle -= 5) { servoPulse(SERVO_PIN, angle);
}
  delay(5000);
  distance_L = Ultrasonic_read();
  Serial.print("D L="); Serial.println(distance_L);
  delay(1000);

  // Return to center (0 -> 70)
  for (int angle = 0; angle <= 50; angle += 5) { servoPulse(SERVO_PIN, angle);
}
  delay(3000);

  compareDistance();
}

// ---------------------------------------------------------
// Compare left vs right distance and maneuver around
// ---------------------------------------------------------
void compareDistance()
{
  if (distance_L > distance_R)
  {
    // More space on left -> turn left and go around
    rotateMotor(MOTOR_SPEED, -MOTOR_SPEED); delay(500);  // turn left
    rotateMotor(MOTOR_SPEED,  MOTOR_SPEED); delay(600);  // forward
    rotateMotor(-MOTOR_SPEED, MOTOR_SPEED); delay(500);  // turn right
```

```
    rotateMotor(MOTOR_SPEED,  MOTOR_SPEED); delay(600);  // forward
    rotateMotor(-MOTOR_SPEED, MOTOR_SPEED); delay(400);  // turn right
  }
  else
  {
    // More space on right -> turn right and go around
    rotateMotor(-MOTOR_SPEED, MOTOR_SPEED); delay(500);  // turn right
    rotateMotor( MOTOR_SPEED, MOTOR_SPEED); delay(600);  // forward
    rotateMotor( MOTOR_SPEED,-MOTOR_SPEED); delay(500);  // turn left
    rotateMotor( MOTOR_SPEED, MOTOR_SPEED); delay(600);  // forward
    rotateMotor( MOTOR_SPEED,-MOTOR_SPEED); delay(400);  // turn left
  }
}

// ----------------------------------------------------------
// Motor control
// positive = forward, negative = backward, 0 = stop
// ----------------------------------------------------------
void rotateMotor(int rightMotorSpeed, int leftMotorSpeed)
{
  if (rightMotorSpeed < 0)
  {
    digitalWrite(rightMotorPin1, LOW);
    digitalWrite(rightMotorPin2, HIGH);
  }
  else if (rightMotorSpeed > 0)
  {
    digitalWrite(rightMotorPin1, HIGH);
    digitalWrite(rightMotorPin2, LOW);
  }
  else
  {
    digitalWrite(rightMotorPin1, LOW);
    digitalWrite(rightMotorPin2, LOW);
  }

  if (leftMotorSpeed < 0)
  {
    digitalWrite(leftMotorPin1, LOW);
    digitalWrite(leftMotorPin2, HIGH);
  }
  else if (leftMotorSpeed > 0)
  {
    digitalWrite(leftMotorPin1, HIGH);
    digitalWrite(leftMotorPin2, LOW);
  }
  else
  {
```

```
    digitalWrite(leftMotorPin1, LOW);
    digitalWrite(leftMotorPin2, LOW);
  }

  analogWrite(enableRightMotor, abs(rightMotorSpeed));
  analogWrite(enableLeftMotor,  abs(leftMotorSpeed));
}
```

**Experimental Procedure:**

1. The robot chassis was assembled by mounting two DC motors and attaching wheels to each motor shaft.
2. The L293D-based motor driver was connected to the Arduino Uno: the enable pins to pins 6 (right motor) and 5 (left motor), and the direction pins to pins 7 and 8 (right motor) and 9 and 10 (left motor). The two DC motors were wired to the driver outputs.
3. The right and left IR sensor modules were connected to pins 11 and 12 respectively; sensitivity was adjusted using the onboard potentiometers until stable black/white detection was confirmed with the indicator LEDs on the sensor modules.
4. The HC-SR04 ultrasonic sensor was mounted on the SG90 servo and wired with TRIG to pin 13 and ECHO to pin 4, and the servo signal was connected to pin 3. A simple distance test sketch was uploaded to verify correct distance readings on the Serial Monitor.
5. The complete program was written in the Arduino IDE (no external library is required), compiled, and uploaded to the Arduino Uno via USB.
6. A black tape track was prepared on a white surface with straight segments and one 90° turn. The robot was placed at the start of the track, the battery pack was connected, and the robot's line-following behaviour was observed and adjusted by tuning IR sensor sensitivity.
7. An obstacle (a cardboard box) was placed on the track. The robot's ability to detect the obstacle and execute the avoidance manoeuvre was tested and verified.

**Results and Discussion:**

The robot successfully followed the black line on a white surface using IR sensors, moving forward when both sensors were over the white surface on either side of the line and turning left or right toward the line when only one sensor reached it. If both sensors detected black at the same time, the robot stopped to avoid leaving the track. The obstacle avoidance system also worked properly; when the distance ahead fell to 5 cm or less, the robot stopped, scanned the right and left sides with the servo-mounted sensor, passed the obstacle on the side with more free space, and resumed line following. The HC-SR04 sensor provided accurate distance measurements without false triggering. Motor speed was set to 110/255 to ensure stable and smooth operation.



**Fig.9.3:** LFR detecting the object and avoiding it

**Conclusion:**

In this experiment, a two-wheel-drive smart robot was successfully designed and constructed, integrating line following and servo-assisted obstacle avoidance. The dual IR sensor system provided reliable line detection, and the HC-SR04 ultrasonic sensor, mounted on a servo, enabled effective short-range obstacle detection and avoidance by comparing the free distance on the left and right sides. The L293D-based motor driver offered straightforward speed and direction control of the DC motors through PWM and direction pins. The experiment demonstrated that sensing and actuation could be successfully integrated on a single Arduino Uno platform within the constraints of available I/O pins and processing resources.

**Reference:**


1. HC-SR04 Ultrasonic Sensor Datasheet, Cytron Technologies.
2. Floyd, T. L. (2012). Electronic Devices: Conventional Current Version (9th ed.). Pearson Education.